CareerHub · Assignment 1.4 · Teaching edition

Step-by-step mastering 1.4

Applications & Mutations — explained so a beginner (or a non-technical person) can follow every step: what to type, which file to open, the exact code, why each important line exists, what it does to the screen, and a real-world scenario for each part.

The big picture in one sentence. We are turning a job board you can only *read* into one you can *write to*: a person fills in a form, the app checks it like a bouncer at a club door, and only if everything is correct does it send the application to a "kitchen" (the server) that saves it and replies. A recruiter can then read those applications and say yes or no.

What you will build, in plain words

- **A form** on the job page where a candidate applies.
- **A checker (Zod)** that catches mistakes *before* anything is sent.
- **A sender (useMutation)** that posts the data and shows loading / success / error.
- **A mock backend** (server routes) that receives, validates again, waits 0.8s, and saves.
- **Extra:** a recruiter area to post jobs, count applicants, read CVs, and accept/reject.

Map of this guide

0 Setup & install

COMMANDS YOU SHOULD RUN

```
cd careerhub-1.3
npm install react-hook-form @hookform/resolvers zod
```

This adds three libraries:

Library	In one sentence
`react-hook-form` (RHF)	Manages the form's typing/values without re-drawing the screen on every keystroke.
`zod`	The "rule book" + bouncer: you describe what valid data looks like and it checks it.
`@hookform/resolvers`	The little translator that lets RHF use Zod's rule book.

THEN, TO RUN THE PROJECT

```
npm run dev  # opens http://localhost:3000
npm run build  # the marking gate: must show 0 errors
```

> **Why a separate install?** The starter project only knew how to *read* jobs. These three packages are the tools for *writing* data safely. We install them once, up front.

1 Part 1 — The four written decisions

> These are **answers you write in** `README.md` — no code to run. They prove you understand *why*, not just *how*.

Decision 1 — Why `@hookform/resolvers` is its own package

Analogy. RHF speaks English, Zod speaks French. Instead of forcing one to learn the other, there is a small **interpreter** in the middle. The interpreter is `@hookform/resolvers`.

Keeping them separate means each library can update on its own and support many partners (RHF works with Zod, Yup, Joi…; Zod is used far beyond forms). At runtime `zodResolver(schema)` returns a function RHF calls on submit. It **receives** the form values, **calls** `schema.safeParseAsync(values)`, and **returns** either `{ values }` (clean, parsed data) or `{ errors }` (mistakes keyed by field name).

Decision 2 — The number-input problem

An `<input type="number">` gives back the text `"5"`, not the number `5`. Zod's `z.number()` refuses text. Two fixes: **(A)** `register("x",{valueAsNumber:true})` converts in the form; **(B)** `z.coerce.number()` converts in the schema. Both end up as a real `number`, so `z.infer` is identical. **We chose B** so every rule lives in one place — the schema.

Decision 3 — `mutate` vs `mutateAsync` (the sneaky bug)

Analogy. `mutate` is posting a letter and walking away — you don't wait for a reply. `mutateAsync` is posting it and *waiting at the postbox* until the reply comes.

The form's "is it busy?" flag (`isSubmitting`) stays true only while the submit function is still running. `mutate` finishes instantly, so the button would wrongly re-enable during the 0.8s wait. `mutateAsync` returns a promise we `await`, so the button stays disabled until the server actually replies. **We use** **`mutateAsync`**.

Decision 4 — Where to put `onSuccess`

Option A (inside `useMutation`) runs after *every* successful submit. **Option B** (passed to one `mutate` call) runs for that one call only. We use **Option A** to refresh the job list (`["jobs"]`) and clear the form, because that should always happen on success, no matter who triggered it.

Summary of Part 1. Four short essays in the README explaining the *reasoning*: separation of libraries, string-vs-number, the timing bug, and callback placement. No UI change — this is the "why" behind everything that follows.

2 Part 3 — The Types (the contract)

> Open the folder `src/types/` → file `index.ts`. Add these to the bottom (don't change existing types).

We do types first because everything else (the route, the sender, the form) refers to them. A "type" is just a **shape** — a promise about which fields exist and what kind of value each one is.

```
export interface ApplicationRequest {
  jobId: string;
  fullName: string;
  email: string;
  phone?: string;            // the ? means optional
  yearsOfExperience: number;
  coverLetter: string;
  linkedInUrl?: string;      // optional
  availableImmediately: boolean;
  noticePeriodWeeks: number;
}

export interface ApplicationResponse {
  id: string;                // the server makes this
  jobId: string;
  email: string;
  submittedAt: string;       // when it was saved
}
```

IMPORTANT LINES EXPLAINED

- `ApplicationRequest` = what the browser **sends**. `ApplicationResponse` = what the server **sends back**.
- `phone?` and `linkedInUrl?` — the **?** says "you may leave this out".
- `number` vs `string` vs `boolean` — TypeScript will refuse to build if we ever put the wrong kind of value in a field. That is a *good* failure: it stops bugs before users see them.

> **How it affects the UI?** Indirectly but powerfully: because the form's data is typed, your editor auto-completes field names and the build *fails loudly* if the form and the server ever disagree — so the UI never silently sends broken data.

Scenario. Six months from now a teammate renames `email` to `emailAddress` on the server but forgets the form. With these types, the project **won't build** — you find the mismatch in seconds instead of from an angry user.

Summary of Part 3. Two shapes describe the application going out and the confirmation coming back. They are the single agreement every other file trusts.

3 Part 2 — The Route (the backend "kitchen")

> Create the folders/file: `src/app/api/applications/route.ts`. In Next.js, a file called `route.ts` inside `app/api/...` automatically becomes a web address.

Analogy. This file is the restaurant kitchen. The form is a customer handing in an order (POST). The kitchen checks the order is complete, takes a realistic amount of time to cook (0.8s), then serves the dish (201) or sends the order back if it's wrong (400).

```
import { NextResponse } from "next/server";
import { addApplication } from "@/lib/server-store";
import type { ApplicationRequest } from "@/types";

// helper: an error in a tidy, standard shape
function problem(title: string, detail: string, status: number) {
  return NextResponse.json({ title, detail, status }, { status });
}

export async function POST(request: Request) {
  // 1) read the order (the JSON the form sent)
  let body: Partial<ApplicationRequest>;
  try { body = await request.json(); }
  catch { return problem("Invalid request body", "Body is not valid JSON.", 400); }

  // 2) the two fields the server insists on
  if (!body.jobId || !body.email) {
    return problem("Missing required fields",
      "Both 'jobId' and 'email' are required to submit an application.", 400);
  }

  // 3) wait 800ms so the loading state is visible in the browser
  await new Promise<void>((resolve) => setTimeout(resolve, 800));

  // 4) save it, then reply 201 (Created) with a small confirmation
  const saved = await addApplication(body as ApplicationRequest);
  return NextResponse.json(
    { id: saved.id, jobId: saved.jobId, email: saved.email, submittedAt: saved.submitted
    { status: 201 });
}

export async function GET() {   // only POST is allowed → GET gets 405
  return NextResponse.json(
    { title:"Method Not Allowed", detail:"This endpoint only accepts POST requests.", st
    { status:405, headers:{ Allow:"POST" } });
}
```

IMPORTANT LINES EXPLAINED

- `export async function POST` — the name **POST** is what makes this run only for "send me data" requests.
- `if (!body.jobId || !body.email)` → `400` — the server double-checks, even though the form already checked. **Never trust the client alone.**

- `setTimeout(resolve, 800)` — a deliberate 0.8-second pause. Without it the request is so fast you'd never see the "Submitting…" state.
- `status: 201` — the official "Created" code. `200` means OK; `201` specifically means "I made a new thing".
- The separate `GET()` returning `405` — politely says "wrong door, use POST".

> **How it affects the UI?** The 0.8s pause is what lets the button show "*Submitting…*". A 400 makes the red error panel appear. A 201 makes the green success panel appear. The backend's replies literally drive what the user sees.

Scenario. A hacker bypasses your form and posts `{}` directly. The server still answers `400` and saves nothing — your data stays clean because validation lives on the server too, not only in the browser.

Summary of Part 2. One file becomes the `/api/applications` address. It accepts POST, rejects GET (405), rejects incomplete orders (400), waits 0.8s, saves, and returns 201 with a confirmation.

4 Part 4 — The API layer (the "sender")

> Open `src/lib/api.ts` and add `submitApplication` next to the existing `fetchJobs`.

Analogy. If the route is the kitchen, this is the **waiter**: it carries the order to the kitchen, and if the kitchen says "no", it brings the exact complaint back to you.

```
const BASE_URL = () => process.env.NEXT_PUBLIC_API_URL ?? "";

export async function submitApplication(
  body: ApplicationRequest,
): Promise<ApplicationResponse> {
  const res = await fetch(`${BASE_URL()}/api/applications`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body),
  });

  if (!res.ok) {                        // not a 2xx? turn it into an error
    const problem = await res.json().catch(() => ({}));
    throw new Error(problem.detail ?? problem.title ?? `Request failed with ${res.status
  }

  return await res.json() as ApplicationResponse;
}
```

IMPORTANT LINES EXPLAINED

- `method: "POST"` + `Content-Type: application/json` + `JSON.stringify(body)` — "I'm sending you data, and it's in JSON format."
- `if (!res.ok)` — `fetch` does **not** throw on a 400/500; it quietly returns them. This line is what turns a server "no" into a real error our form can react to.
- `throw new Error(problem.detail ?? problem.title …)` — we surface the *server's own words* ("Both jobId and email are required") instead of a generic message.
- No React here on purpose — this is pure logic, easy to test and reuse.

> **How it affects the UI?** When this function *throws*, the form's mutation flips to "error" and shows the red panel with that exact message. When it returns normally, the green success panel shows.

Scenario. The server is down. `res.ok` is false, we throw a clear message, and the candidate sees "couldn't submit" instead of a frozen button or a blank page.

Summary of Part 4. A tiny, pure function that POSTs the application and either returns the confirmation or throws the server's error message. This is the `mutationFn` the form will hand to TanStack Query.

5 Part 5 — The ApplicationForm

> Create `src/components/ApplicationForm.tsx`. This is the biggest piece — the actual screen the candidate uses.

5a. The rule book (Zod schema)

> **Analogy.** The schema is the **bouncer's checklist** at the door. Each line is one rule. If any rule fails, the bouncer turns the data away *before* it reaches the street (the network).

```
const phoneRegex = /^\+?[\d\s\-()]{8,15}$/;

const applicationSchema = z.object({
  fullName: z.string().min(2, "...at least 2 characters.").max(100, "...."),
  email: z.email("Enter a valid email address."),

  // optional: a valid phone OR a blank box ("") — blank is allowed
  phone: z.string().regex(phoneRegex, "Enter a valid phone number.")
          .or(z.literal("")).optional(),

  // the box gives "5" (text); coerce turns it into the number 5
  yearsOfExperience: z.coerce.number().int().min(0).max(50),

  coverLetter: z.string()
    .min(50, "Cover letter must be at least 50 characters — tell us why you're a strong
    .max(2000, "...."),

  linkedInUrl: z.url("Enter a valid URL.")
    .refine(v => v.includes("linkedin.com"), "Must be a LinkedIn URL.")
    .or(z.literal("")).optional(),

  availableImmediately: z.boolean(),
  noticePeriodWeeks: z.coerce.number().int().min(0),
})
// a rule that looks at TWO fields at once:
.refine((data) => data.availableImmediately || data.noticePeriodWeeks > 0, {
  message: "If you're not available immediately, your notice period must be at least 1 w
  path: ["noticePeriodWeeks"],   // show the error on this field
});

type ApplicationFormData  = z.infer<typeof applicationSchema>;   // the validated shape
type ApplicationFormInput = z.input<typeof applicationSchema>;   // the raw-typed shape
```

THE IMPORTANT IDEAS

- `.or(z.literal(""))` — the magic for optional boxes. An empty text box sends `""` (not "nothing"), which would normally fail the phone/URL rule. This says "a valid value *or* an empty box is fine". Without it, leaving phone blank would show a false error.
- `z.coerce.number()` — converts the text from a number box into a real number, so the rules `.int().min().max()` work.

- `.refine( ... )` **with** `path` — a cross-field rule: if you are *not* available now, you must give a notice period. `path:["noticePeriodWeeks"]` makes the red message appear on that exact box. A normal `.min(1)` couldn't do this, because it would force a notice period on *everyone*, even people available immediately.
- `z.infer` — we never hand-write the data type; we derive it from the rules so the two can never disagree.

5b. The form logic (RHF + the mutation)

```
const { register, handleSubmit, reset, formState:{ errors, isSubmitting } } =
  useForm<ApplicationFormInput, unknown, ApplicationFormData>({
    resolver: zodResolver(applicationSchema),
    defaultValues: { availableImmediately:true, noticePeriodWeeks:0, yearsOfExperience:0
  });

const mutation = useMutation({
  mutationFn: submitApplication,
  onSuccess: () => {                                    // Option A - runs on every succes
    queryClient.invalidateQueries({ queryKey:["jobs"] });   // refresh the board
    reset();                                            // clear the form
  },
});

const onValid = async (data) => {
  const request = { jobId,  ...data,
     ...(data.phone ? { phone: data.phone } : {}),       // drop empty optionals
     ...(data.linkedInUrl ? { linkedInUrl: data.linkedInUrl } : {}) };
  try { await mutation.mutateAsync(request); }          // WAIT for the reply
  catch { /* shown via mutation.isError */ }
};

const isBusy = isSubmitting || mutation.isPending;     // two flags, one guard
```

IMPORTANT LINES EXPLAINED

- `register` connects each input to the form. `handleSubmit(onValid)` runs Zod first and only calls `onValid` if **every** rule passes.
- `defaultValues` — the checkbox starts ticked (available now), numbers start at 0, so the form is valid-by-default and friendly.
- `await mutation.mutateAsync( ... )` — the fix from Decision 3: we *wait*, so the button stays disabled for the whole 0.8s.
- `isBusy = isSubmitting || mutation.isPending` — two "busy" signals OR-ed together so the button is disabled from the first click until the reply, with no gap in the middle.

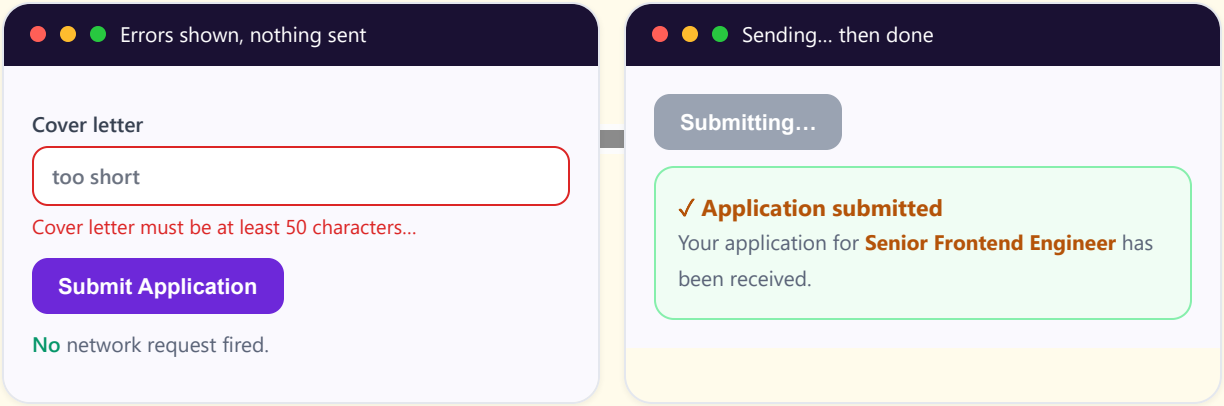
5c. The screen (the JSX) — one field, the pattern repeats

```
<form onSubmit={handleSubmit(onValid)} noValidate>     // noValidate: Zod is the only ju
  ...
  <label htmlFor="fullName">Full name</label>
  <input id="fullName"
         aria-invalid={!!errors.fullName}              // tells screen-readers it's wron
         className={cn(baseInput, errors.fullName ? errBorder : okBorder)}  // red on er
         {...register("fullName")} />                  // no rules here - all rules in s
  {errors.fullName && <p>{errors.fullName.message}</p>}  // the message under the box
  ...
  <button type="submit" disabled={isBusy}
    className={cn(base, isBusy ? "bg-slate-400 cursor-not-allowed" : "bg-brand-600")}>
    {isBusy ? "Submitting…" : "Submit Application"}
```

```
  </button>
</form>
```

- `noValidate` — turns off the browser's own pop-up checks so we get **one** consistent set of rules and messages (ours), not two competing ones.
- `aria-invalid` + red border via `cn` — the error is shown three ways: colour, a message, and for screen-readers. That's accessibility.
- The button shows **"Submitting…"** and turns grey while busy — visible, not just faded.

What the user sees



Scenario. A nervous applicant clicks Submit twice. Because the button is `disabled={isBusy}` during the wait, the second click does nothing — no duplicate application is created.

Summary of Part 5. One component holds the rule book (Zod), the form brain (RHF), and the sender (useMutation). It blocks bad data, shows red field errors, disables + relabels the button while sending, and ends in a green success panel or a red server-error panel.

6 Part 6 — Wiring it into the page

> Open `src/app/page.tsx`. Import the form and render it under the selection panel.

```
import ApplicationForm from "@/components/ApplicationForm";
...
{selectedJob !== null ? (
  <div className="mb-6">
    <ApplicationForm jobId={selectedJob.id} jobTitle={selectedJob.title} />
  </div>
) : null}
```

- This sits inside the "data loaded successfully" branch, so the rule is exactly `!isPending && !isError && selectedJob !== null`.
- We pass the selected job's `id` and `title` down as props, so the form knows which job is being applied for.
- The original selection panel stays — the form is **added below** it, not a replacement.

> **How it affects the UI?** Click a job → the form appears underneath it. Click the same job again (deselect) → the form disappears. No page reload, ever.

Summary of Part 6. Three lines connect the new form to the existing page so it shows up for the selected job, with the job's id/title handed in as props.

7 Extra A — The recruiter database (API)

> New file `src/lib/server-store.ts` + new routes under `src/app/api/`.

Analogy. So far the kitchen cooked and forgot. Now we add a **filing cabinet** (a small database saved to a file) so applications and recruiter-posted jobs are remembered, counted, and can be updated later.

7a. The store — the filing cabinet

```
export type RecruiterDecisionStatus = "Submitted" | "Under review" | "Accepted" | "Rejec
export interface RecruiterApplication extends ApplicationRequest {
  id: string; submittedAt: string; status: RecruiterDecisionStatus;
}

export async function addApplication(body) {                  // candidate applies
  const record = { ...body, id: randomUUID(),
                   submittedAt: new Date().toISOString(), status: "Submitted" };
  store.applications.unshift(record); await persist(store); return record;
}

export async function updateApplicationStatus(id, status) {   // recruiter decides
  const app = store.applications.find(a => a.id === id);
  if (!app) return null;
  app.status = status; await persist(store); return app;       // saved → durable
}
```

- `status` starts at **"Submitted"**. The recruiter later changes it to Accepted/Rejected.
- `persist(store)` writes to a JSON file (`/.data`), so a refresh doesn't lose anything.
- Every application gets a unique `id` (from `randomUUID()`) so it can be found and updated.

7b. The recruiter routes

File (folder)	Address	What it does
`api/recruiter/jobs/route.ts`	POST / GET	Recruiter creates a job / lists their jobs.
`api/jobs/route.ts`	GET	The public board = recruiter jobs + seed jobs, each with a live applicant count.
`api/applications/list/route.ts`	GET	All applicants for one job (`?jobId=…`).
`api/applications/[id]/route.ts`	PATCH	Change one application's status — the Accept/Reject decision.

```
// PATCH /api/applications/[id] — the decision endpoint
export async function PATCH(request, { params }) {
  const { id } = await params;
  const { status } = await request.json();
```

```
  if (!VALID.includes(status)) return problem("Invalid status", "…", 400);
  const updated = await updateApplicationStatus(id, status);
  if (!updated) return problem("Application not found", "…", 404);
  return NextResponse.json(updated);      // the change is now permanent
}
```

> **How it affects the UI?** Posting a recruiter job makes it show up on the home board immediately. The live count is what the candidate's success (which refreshes `["jobs"]`) makes tick upward.

Scenario. A recruiter posts "QA Engineer". A candidate applies. The recruiter refreshes and sees "1 applicant", opens it, clicks Accept — and that decision is saved so it's still "Accepted" tomorrow.

Summary of Extra A. A file-based database plus four routes give the project a real memory: jobs are stored and shown on the board, applications are counted, listed, and their status can be updated and stays updated.

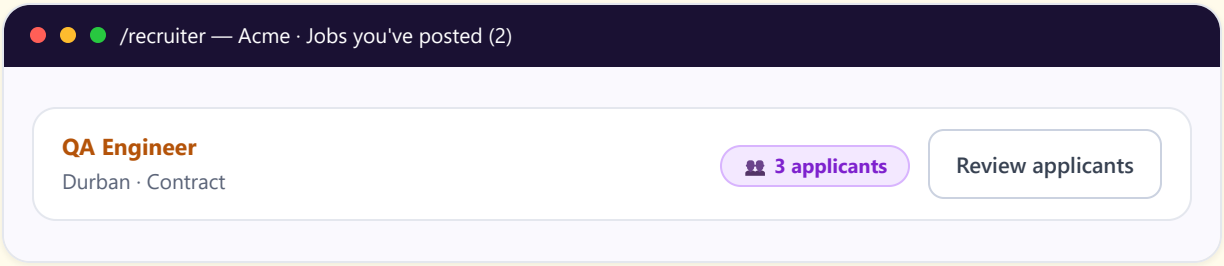
8 Extra B — The recruiter UI & decisions

> Files: `src/app/recruiter/page.tsx` (dashboard) and `src/app/recruiter/jobs/[jobId]/page.tsx` (applicant review).

8a. Post a job & see live counts

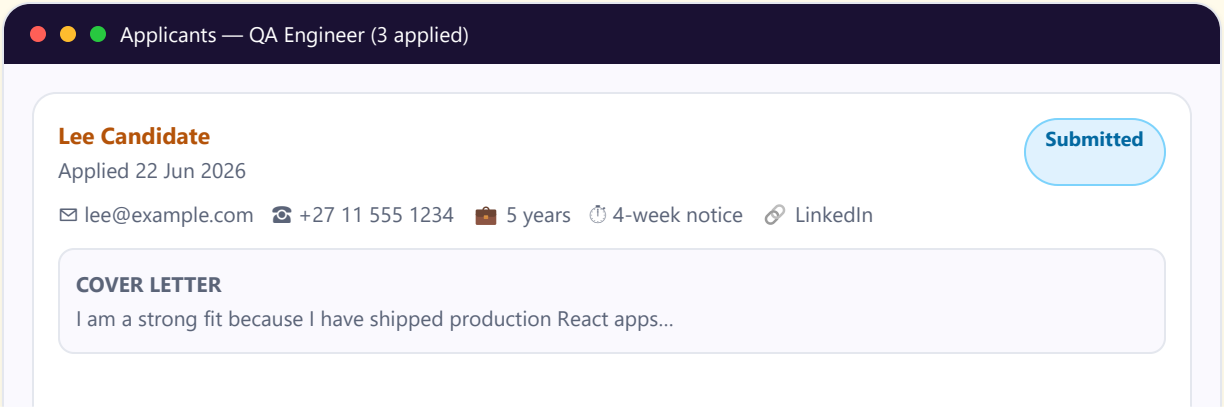
The dashboard uses a mutation to POST the job, then refreshes both the recruiter list and the public board. Each job shows a count badge and a "Review applicants" link.

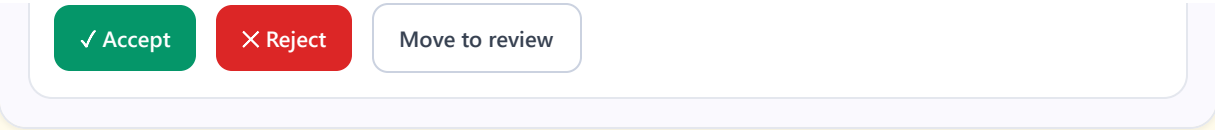
```
const postJob = useMutation({
  mutationFn: postRecruiterJob,
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey:["recruiter-jobs"] });  // my list
    queryClient.invalidateQueries({ queryKey:["jobs"] });            // public board
  },
});
```



8b. Read the CV/details & accept or reject

```
const decide = useMutation({
  mutationFn: ({ id, status }) => decideApplication(id, status),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey:["applicants", jobId] });  // re-read truth
    queryClient.invalidateQueries({ queryKey:["all-applications"] });
  },
});
 …
<button onClick={() => decide.mutate({ id: app.id, status:"Accepted" })}>Accept</button>
<button onClick={() => decide.mutate({ id: app.id, status:"Rejected" })}>Reject</button>
```





- The recruiter sees the full person: contact, experience, availability, LinkedIn, and the cover letter (their pitch / CV).
- Accept/Reject sends a PATCH; on success we **re-read** the list from the server so the badge shows the new status — the screen always matches the database.

Scenario (the full loop). Candidate applies on the home page → it's saved → recruiter opens the job, reads the CV, clicks **Accept** → the status becomes "Accepted" in the database → anyone who looks again sees "Accepted". The two sides of the marketplace are connected.

Summary of Extra B. Two recruiter screens: one to post jobs and watch counts, one to read each applicant and accept/reject. Every decision is saved through the API, so the UI and the database always agree.

9 Proving it works

COMMANDS

```
npm run build  # must say 0 errors — this is the grade gate
npm run dev  # then click around at localhost:3000
```

CHECKLIST (ALL CONFIRMED ON THE RUNNING APP)

Test	Expected	Result
Click Submit on an empty form	Red field errors, **no** network request	Pass
Not available + notice 0	Cross-field error on notice period	Pass
Valid submit	"Submitting…" ~0.8s → green success with job title	Pass
`GET /api/applications`	405	405 (57 ms)
POST missing email	400 with a `detail`	Pass
POST valid body	201 after ~800 ms	1.13 s incl. overhead
Recruiter Accept	Status becomes "Accepted" and stays	Pass
`npm run build`	0 TS + 0 ESLint errors	Clean

You have mastered 1.4 when you can explain: why validation runs in the browser *and* the server, why we wait for the reply (`mutateAsync`), why two busy-flags guard one button, how an empty box (`""`) is handled, and how a recruiter's Accept travels all the way to the database and back to the screen.